



XDP_CLONE: Efficient In-Driver Packet Duplication for eBPF-Accelerated Systems

Marco Mazzella

University of Rome Sapienza

Vladimiro Paschali

University of Rome Sapienza

Francesco Fazzari

University of Rome Sapienza

Beihao Zhou

University of Waterloo

Daniele De Sensi

University of Rome Sapienza

Salvatore Pontarelli

University of Rome Sapienza

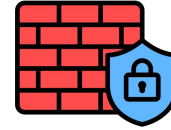
Introduction: packet duplication



HPC
Collective



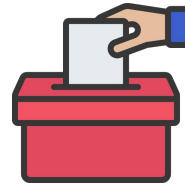
Distributed system
Publish-subscribe brokers



Firewall
Traffic analysis



Database
replication



Distributed consensus
multipaxos

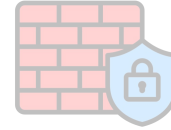
Introduction: case studies



HPC
Collective



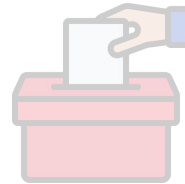
Distributed system
Publish-subscribe brokers



Firewall
Traffic analysis



Database
replication



Distributed consensus
multipaxos

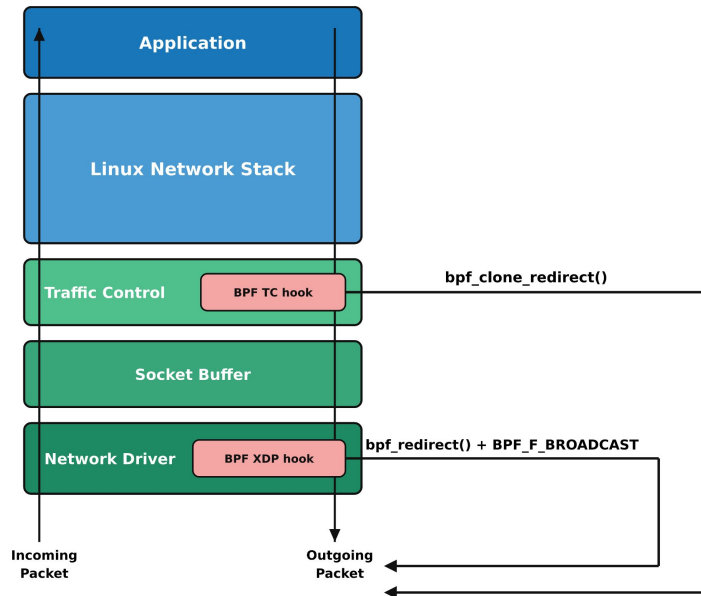
State of the art: eBPF packet duplication

TC using `bpf_clone_redirect()`

- **Heavy fixed cost:** TC runs post-`sk_buff` allocation
- **Per-copy cost:** Each copy requires a new `sk_buff`

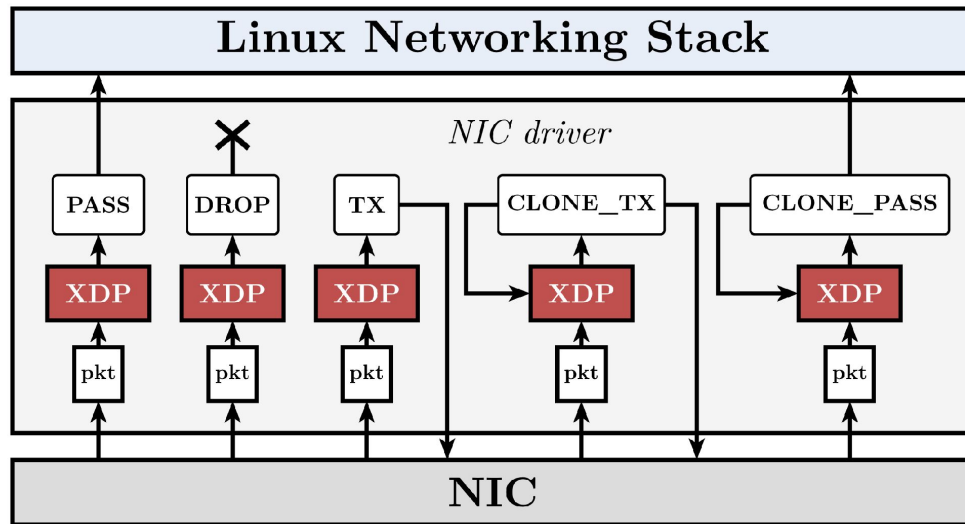
XDP `bpf_redirect_map()` with `BPF_F_BROADCAST`

- **Fixed fan-out** (frame flooding)
- **chain of XDP program**



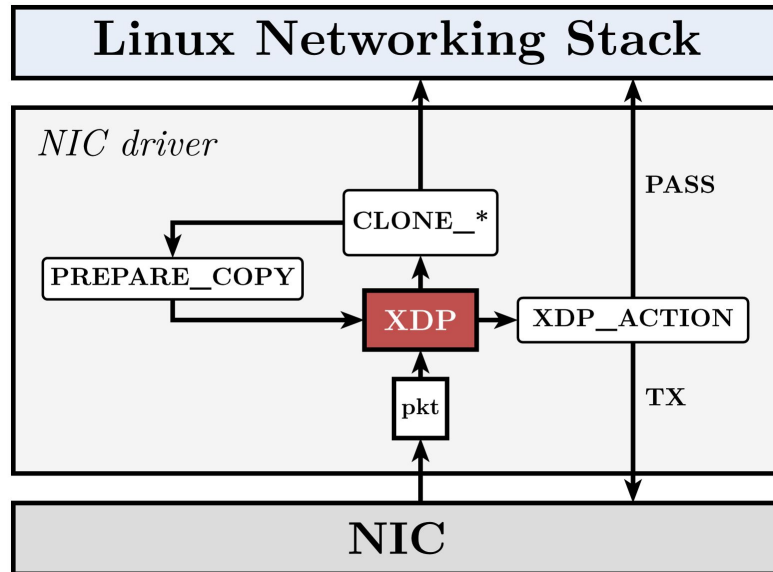
State of the art: Implementation XDP_CLONE

- **New Return Actions:**
`XDP_CLONE_PASS(n)` and
`XDP_CLONE_TX(n)`
- **Original Packet Handled First:**
Delivered to stack (PASS) or
sent on wire (TX)
- **Zero `sk_buff` Overhead:**
Duplication uses in-driver page
pool memory
- **Independent Execution:** Each
clone re-runs XDP logic for
custom header rewrites



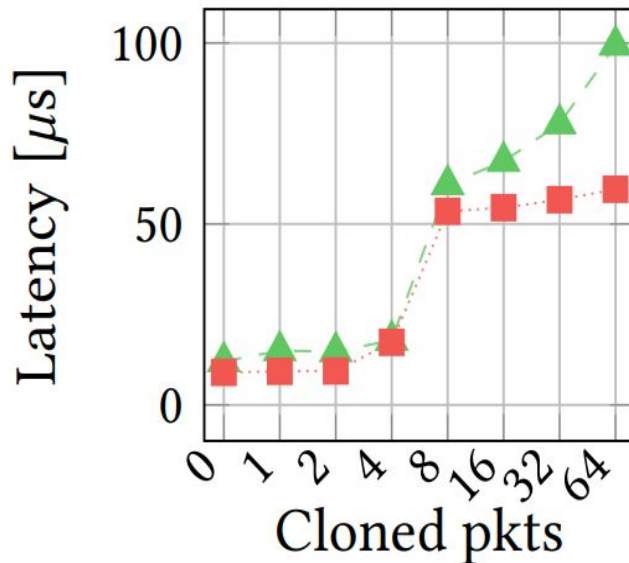
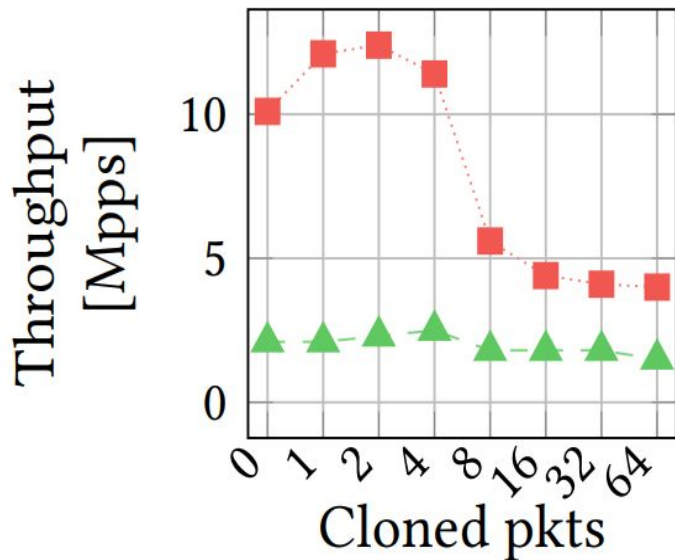
State of the art: In-Driver XDP Packet Cloning & Dispatch

- **Dispatch** each clone still resolves to a **standard action**: PASS, TX, REDIRECT, or DROP
- **Metadata tagging** driver writes clone index n into `data_meta`
- **Safety** nested cloning is blocked (explicit `XDP_ABORTED`)



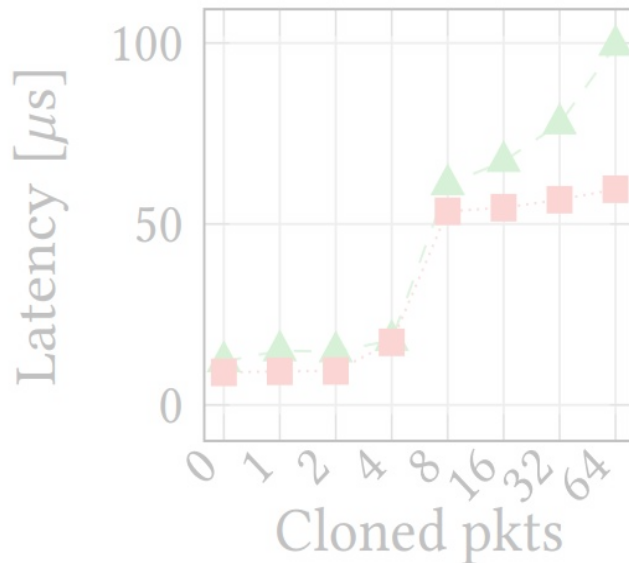
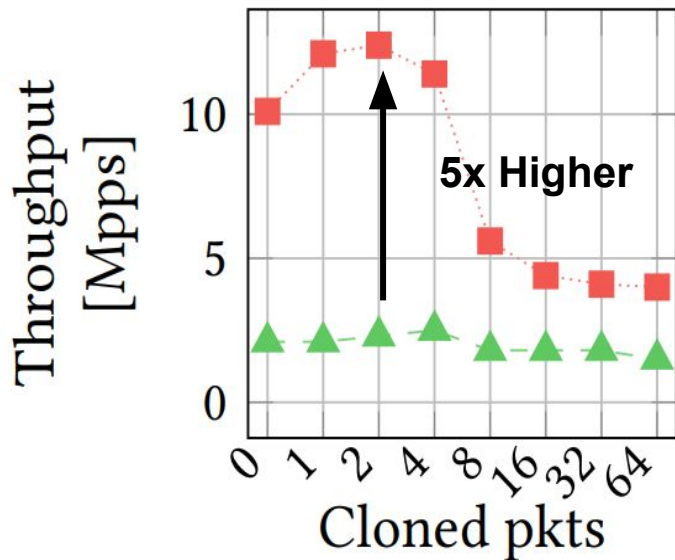
State of the art: Micro benchmark

▲ TC
■ XDP_CLONE

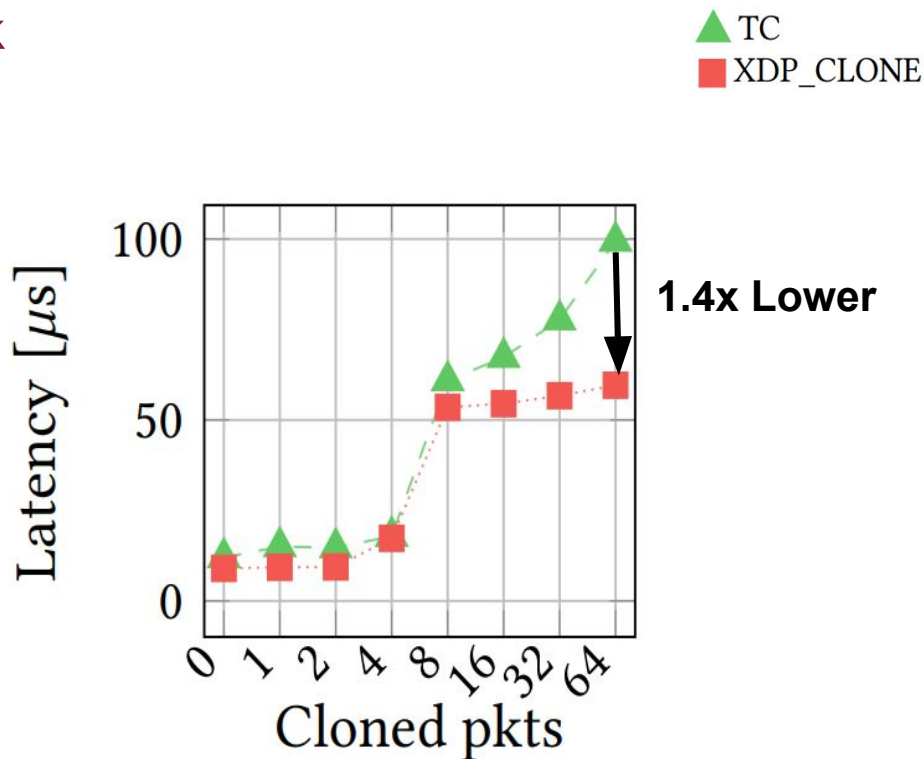
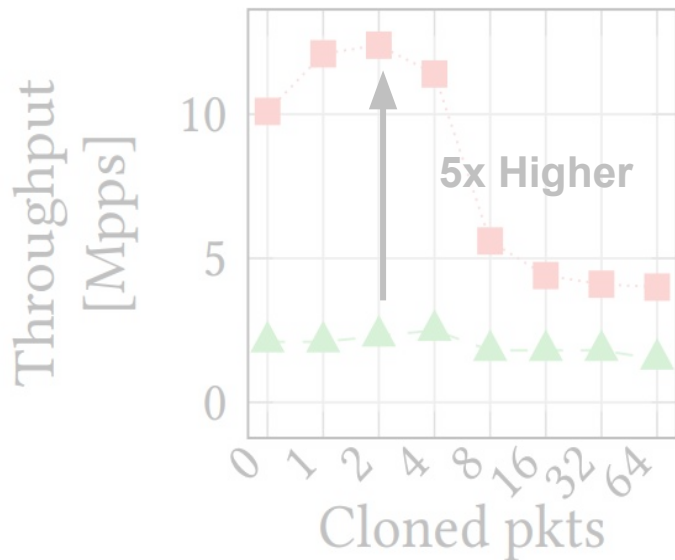


State of the art: Micro benchmark

▲ TC
■ XDP_CLONE

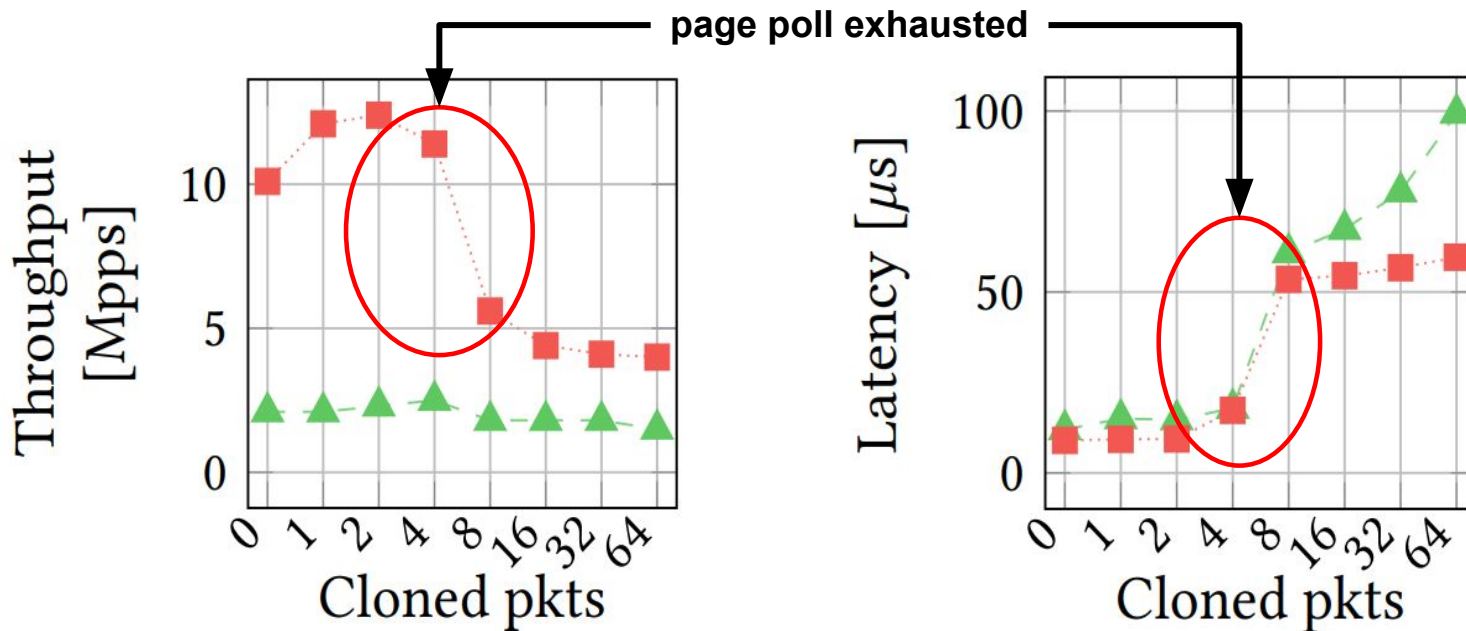


State of the art: Micro benchmark



State of the art: Micro benchmark

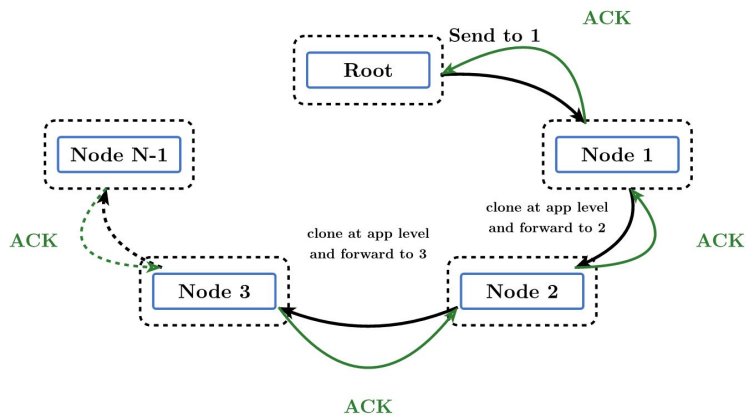
▲ TC
■ XDP_CLONE



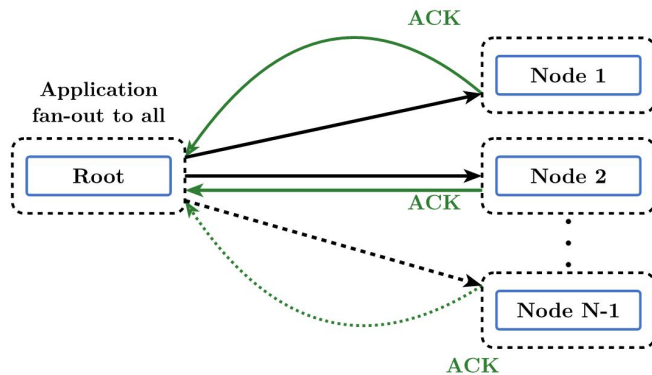
First use case: MPI broadcast

- MPI is a standard** defining **APIs and behaviors** for communication in **distributed/parallel systems**, widely used in **HPC and distributed applications**.

Ring broadcast

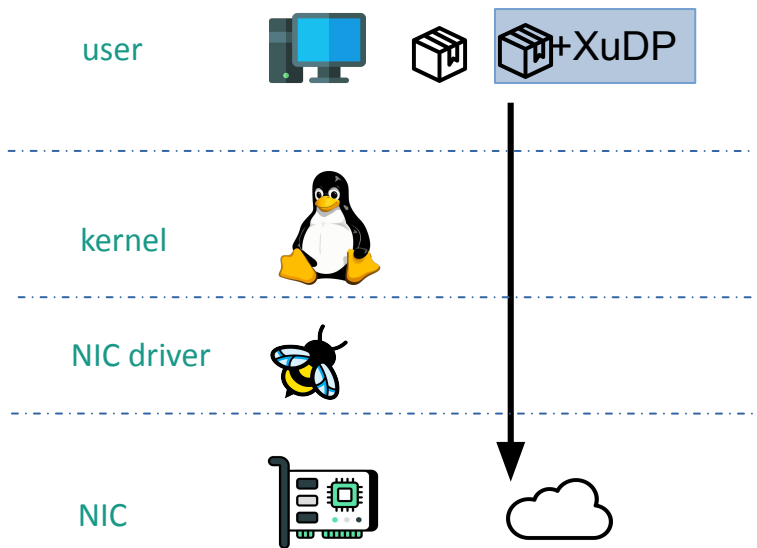


Linear Broadcast



First use case: XuDP

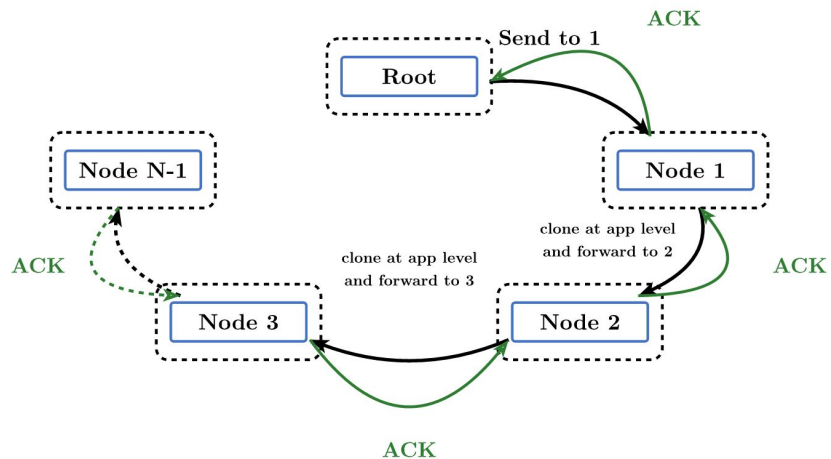
- XuDP** is a lightweight **transport protocol** that combines **UDP** with **XDP_CLONE** to push the **broadcast-forwarding logic** designed for **MPI environments**



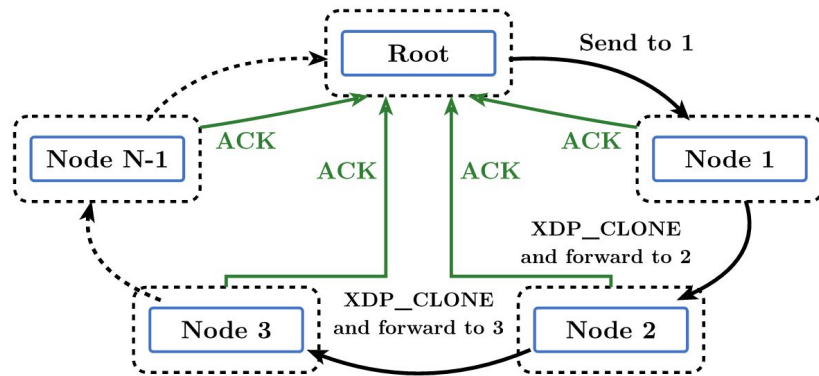
MPI_HDR	Root process
Source process	Destination process
MPI Collective	MPI Datatype
Len	Tag
Sequence number	
ID number	
Data ...	

First use case: XuDP broadcast ring

MPI broadcast ring

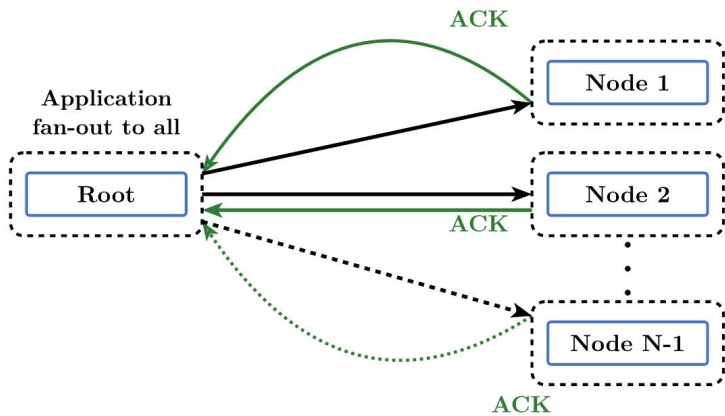


XuDP Broadcast ring

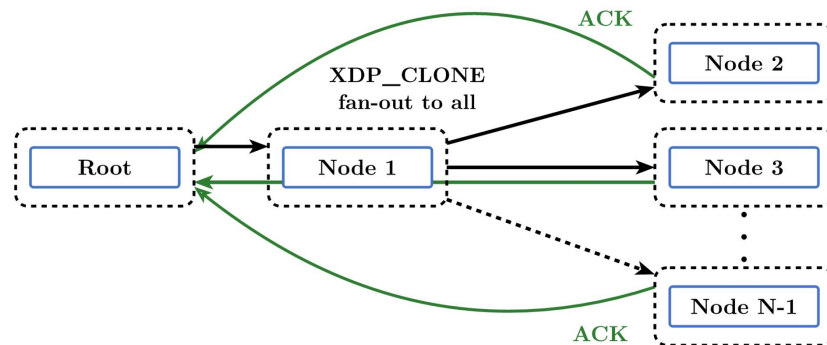


First use case: XuDP broadcast linear

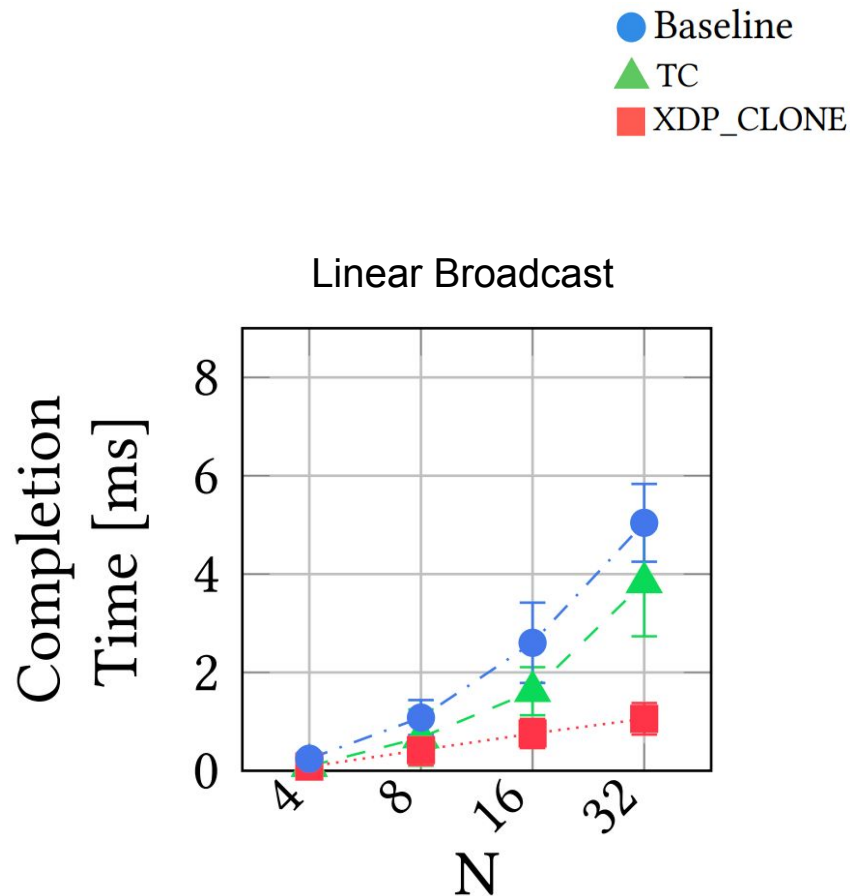
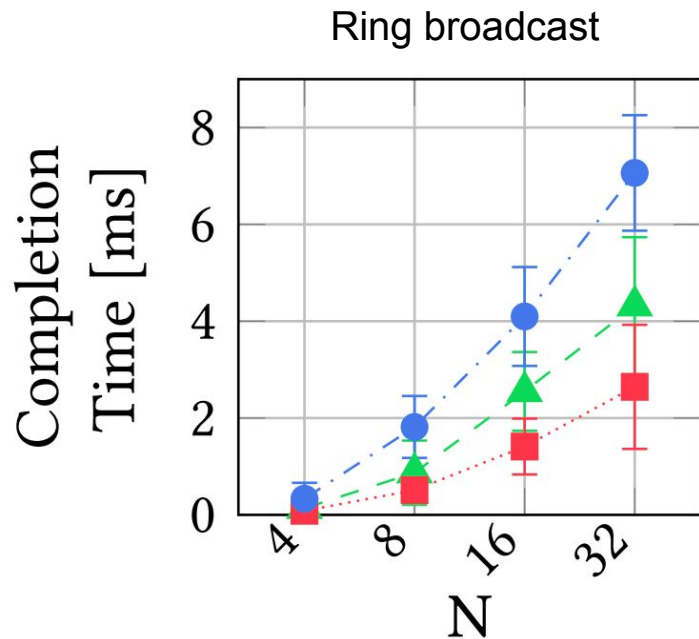
MPI broadcast linear



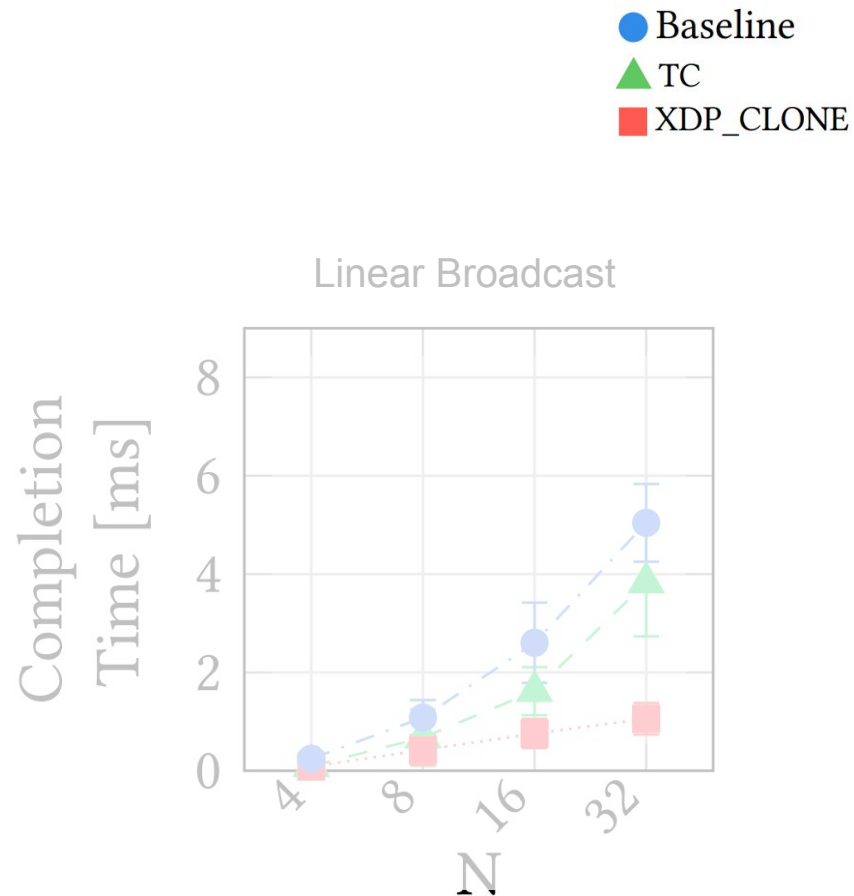
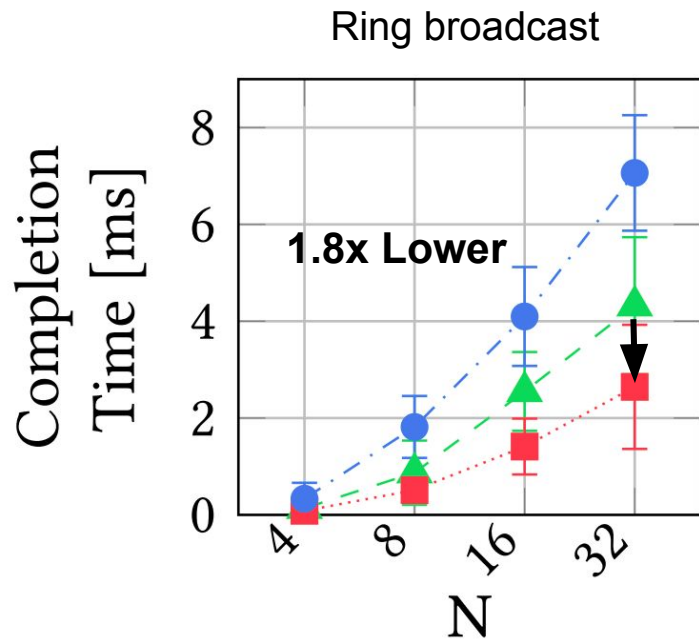
XuDP Broadcast linear



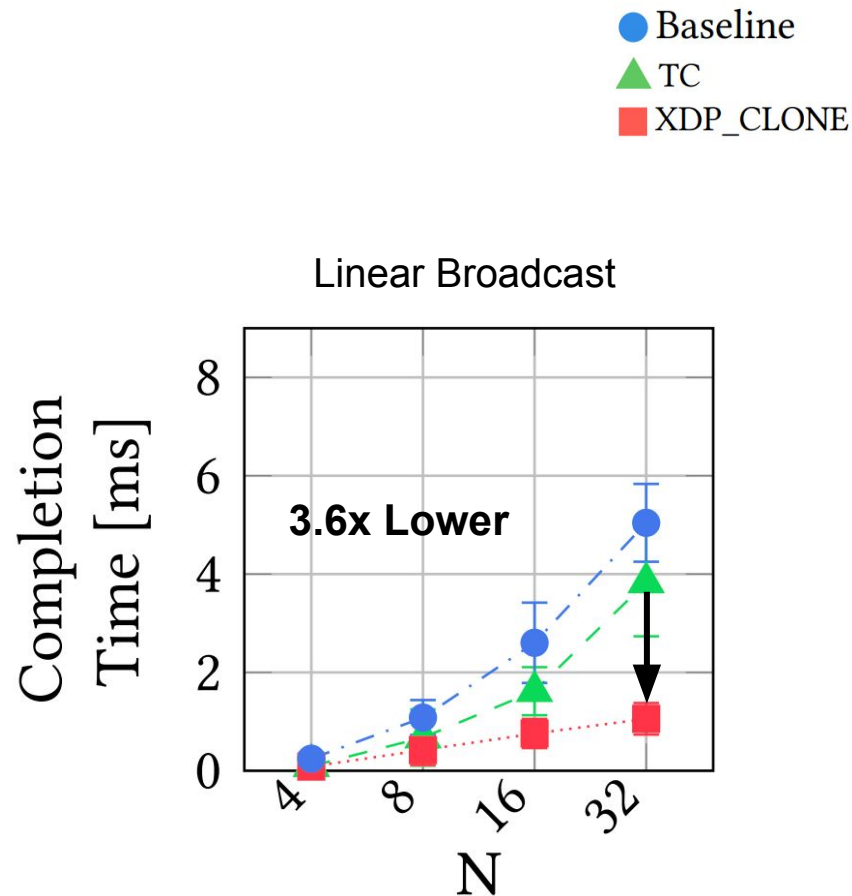
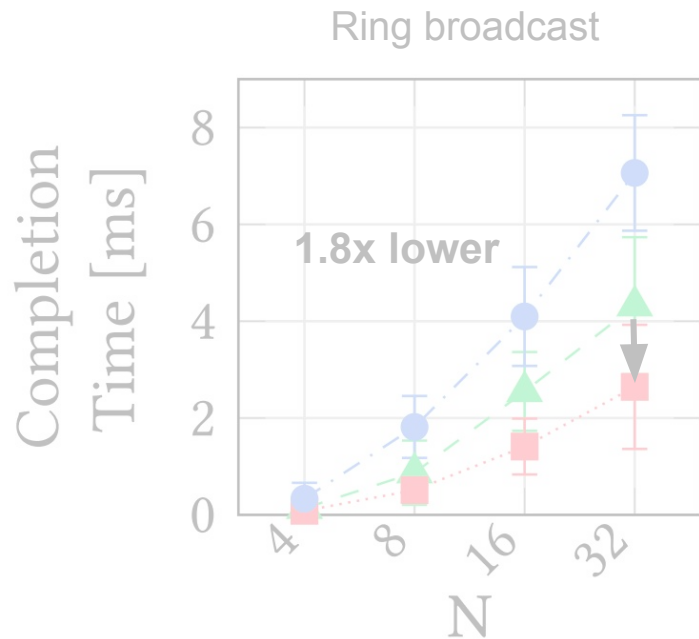
First use case: Result



First use case: Result

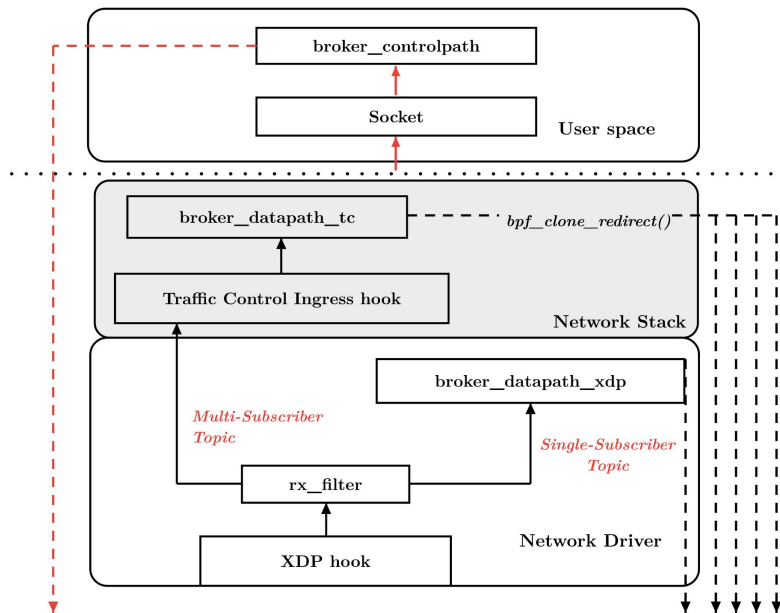


First use case: Result



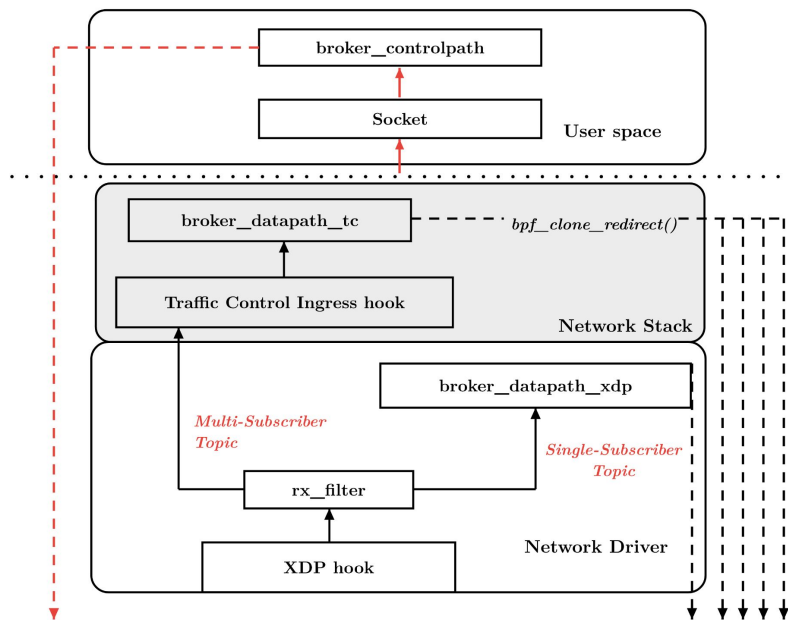
Second use case: BPF-Broker with TC

- We started from **BPF-Broker**, an eBPF-accelerated broker presented at the eBPF workshop last year
- A **pub-sub broker** enables **one-to-many** messaging
- **Subscribers register topics** with a **broker**, which **forwards** each published **message** to all subscribers of that **topic**.

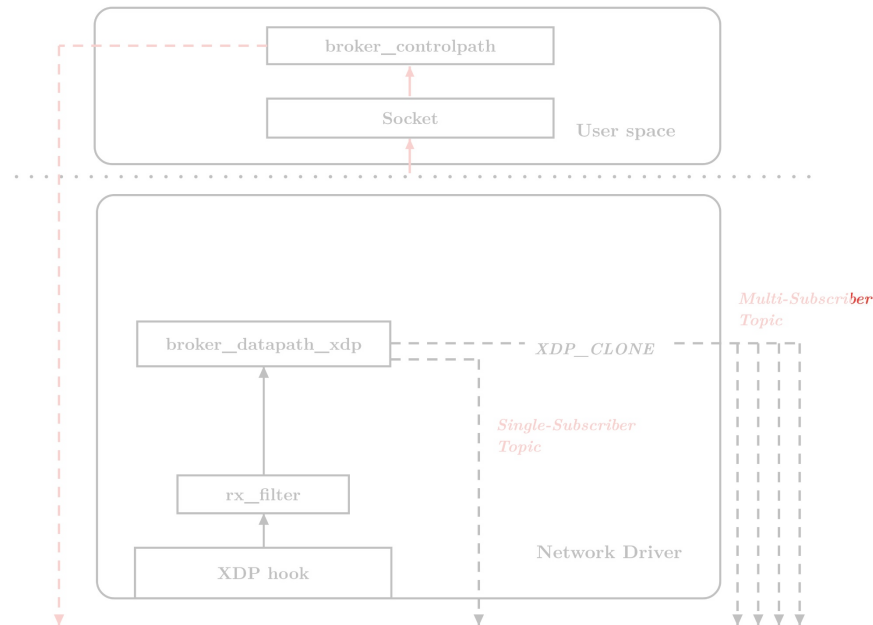


Second use case: BPF-Broker with XDP_CLONE

TC

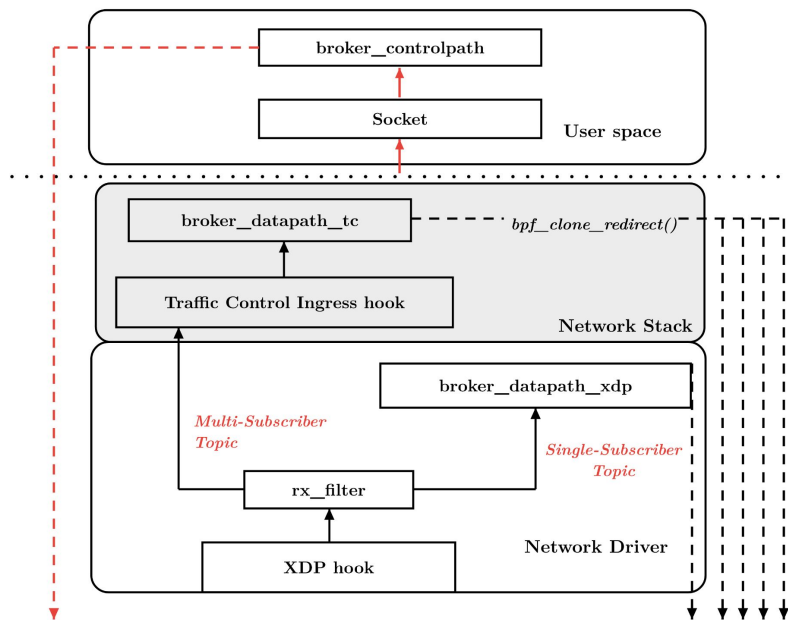


XDP

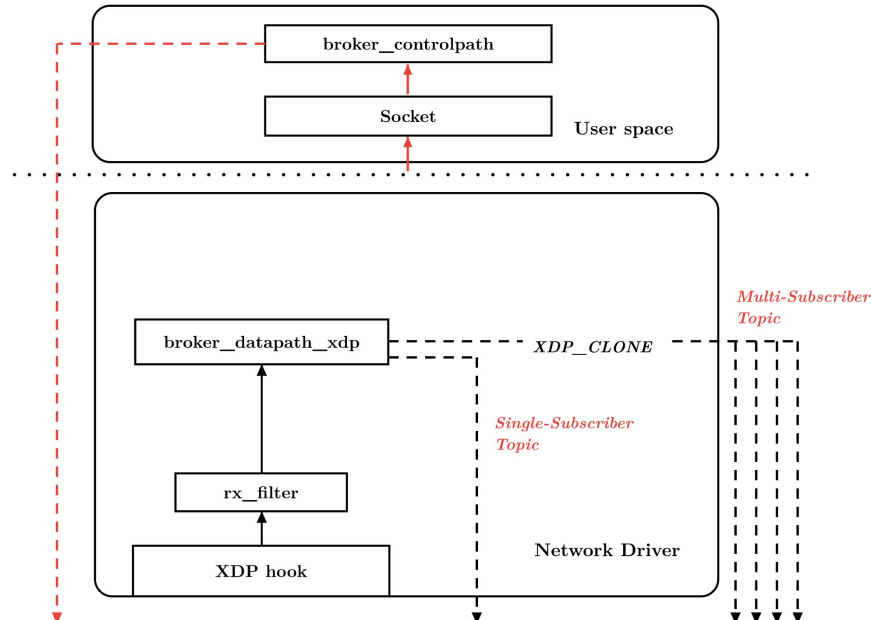


Second use case: BPF-Broker with XDP_CLONE

TC

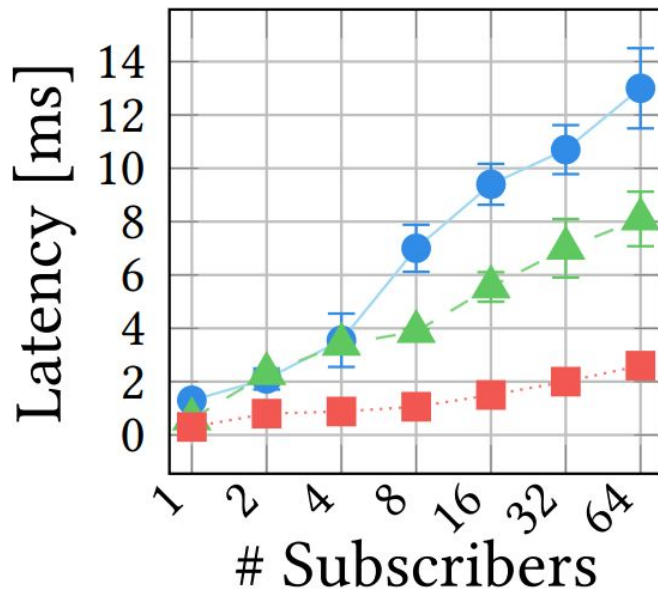
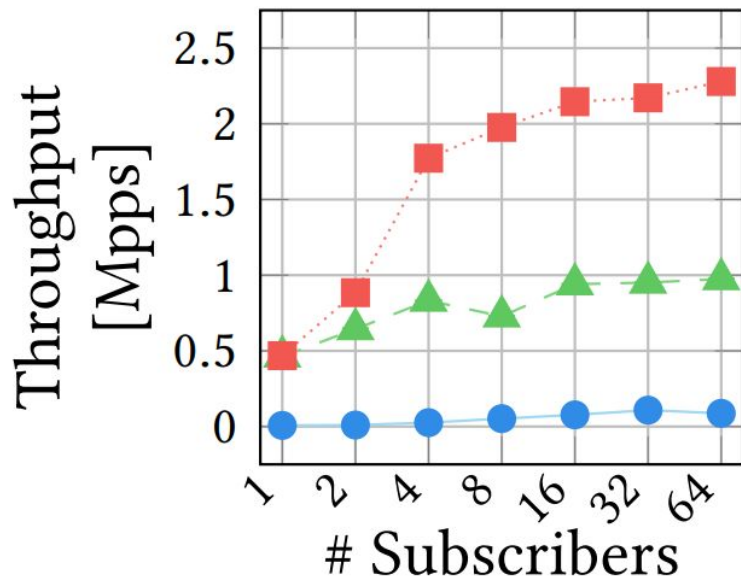


XDP



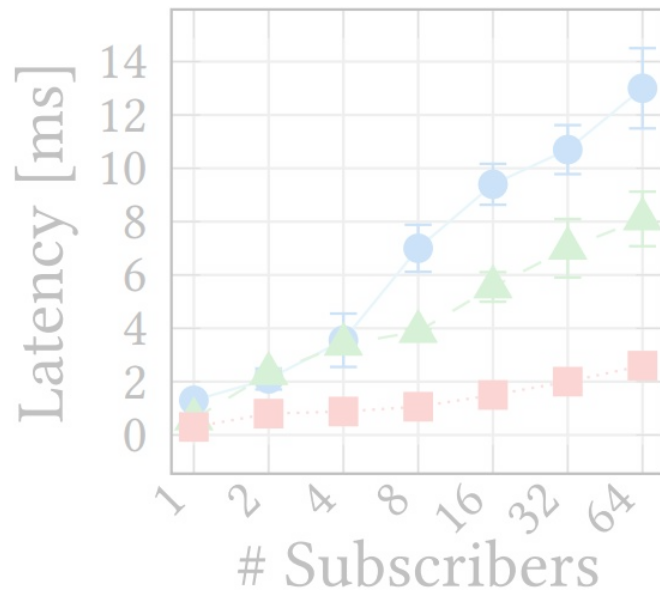
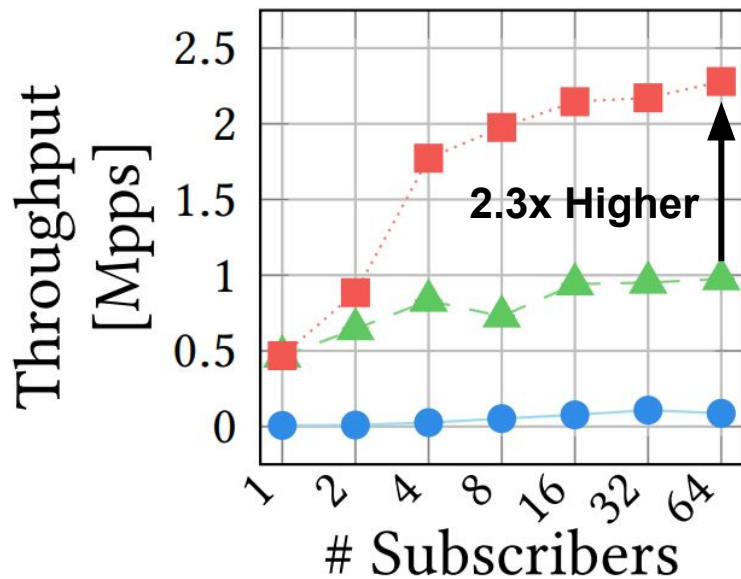
Second use case: BPF-Broker result

- Baseline
- ▲ TC
- XDP_CLONE



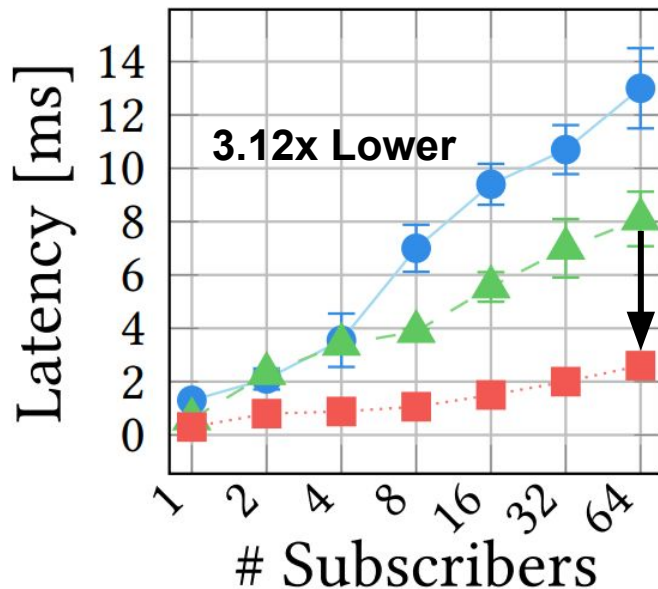
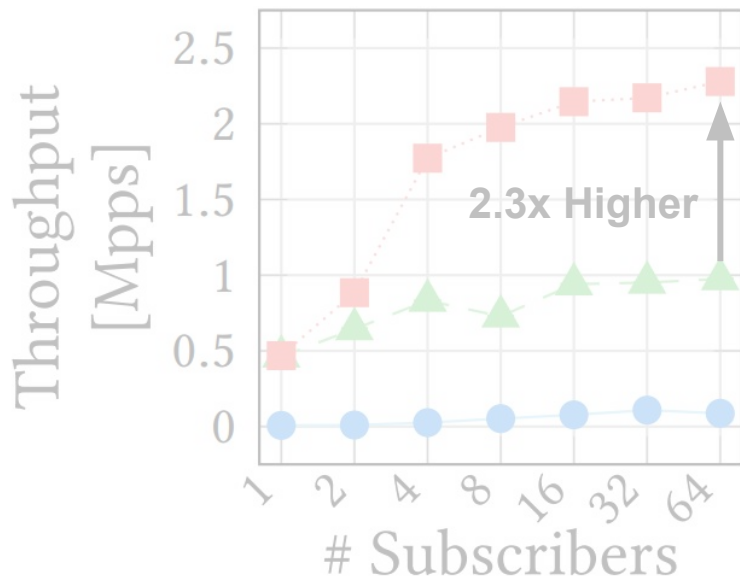
Second use case: BPF-Broker result

- Baseline
- ▲ TC
- XDP_CLONE



Second use case: BPF-Broker result

- Baseline
- ▲ TC
- XDP_CLONE



Conclusions

What we introduce

- **XDP_CLONE**: in-driver, one-to-many packet duplication
- Adds **XDP_CLONE_PASS** / **XDP_CLONE_TX** actions for per-copy decisions

Further Optimizations

- **hardware offloading memcpy in TX**: delegate the actual copy in transmit (TX) to the NIC
- **modify the eBPF verifier**: check statically and at run time XDP_CLONE (i.e. nested XDP_CLONE and return value from a map)

State of the art: Micro benchmark code

```
1 SEC("tc")
2 int tc_clone(struct __sk_buff *skb) {
3     // ... parse Ethernet/IP/UDP, validate ...
4     // One helper call (and one sk_buff allocation) per
   clone
5     for (int i = 0; i < n_clone; i++)
6         bpf_clone_redirect(skb, skb->ifindex, 0);
7     // Forward the original
8     return bpf_redirect(skb->ifindex, 0);
9 }
```

```
1 SEC("xdp")
2 int xdp_clone(struct xdp_md *ctx) {
3     void *data      =(void *) (long) ctx->data;
4     void *data_end  =(void *) (long) ctx->data_end;
5     void *data_meta=(void *) (long) ctx->data_meta;
6     // Clone packet path
7     if (data_meta + sizeof(__u32) <= data) {
8         // Clone ID extraction
9         __u32 num_copy = *(__u32 *) data_meta;
10        if (num_copy > 0 && num_copy <= n_clone)
11            return XDP_TX;    // per-clone action
12    }
13    // Original packet path
14    return XDP_CLONE_TX(n_clone);
15 }
```